Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

4η Εργαστηριακή Άσκηση

Εργασία για High Level Synthesis σε FPGA

Μάθημα: Σχεδιασμός Ενσωματωμένων Συστημάτων

Εξάμηνο: 9ο

Ονοματεπώνυμα: Μιχαλακόπουλος Δημήτριος (ΑΜ: 03120229), Γκενάκου Ζωή (ΑΜ: 03120015)

Εισαγωγή

Η παρούσα εργασία επικεντρώνεται στη βελτιστοποίηση μιας εφαρμογής ανακατασκευής εικόνων, η οποία βασίζεται στη χρήση Generative Adversarial Networks (GANs) και εκτελείται στο σύστημα Zynq-7000 ARM/FPGA SoC (Zybo). Η εφαρμογή χρησιμοποιεί ένα εκπαιδευμένο νευρωνικό δίκτυο generator του GAN για να ανακατασκευάσει το κάτω μισό εικόνων χειρόγραφων αριθμών από το MNIST dataset, λαμβάνοντας ως είσοδο μόνο το πάνω μισό τους. Το τελικό αποτέλεσμα καταγράφεται σε ένα αρχείο (output.txt), το οποίο περιέχει τις τιμές των pixels που παράγονται από το GAN, σε κλίμακα από -1 έως 1.

Στόχος της εργασίας είναι η επιτάχυνση της εφαρμογής με τη χρήση τεχνικών High-Level Synthesis (HLS), ώστε να αξιοποιηθεί το FPGA υποσύστημα του Zybo για την εκτέλεση της πιο απαιτητικής υπολογιστικά συνάρτησης της εφαρμογής, της **forward_propagation**. Η αξιολόγηση περιλαμβάνει τη μέτρηση του αριθμού των κύκλων ρολογιού που απαιτούνται για την εκτέλεση τόσο σε software όσο και σε hardware, καθώς και τον υπολογισμό του **speedup** (Software/Hardware). Ιδανικά, η εφαρμογή στο hardware θα πρέπει να επιτυγχάνει σημαντική μείωση του χρόνου εκτέλεσης και speedup μεγαλύτερο του 1.

Εφαρμογή

Η εφαρμογή υλοποιεί μια διαδικασία ανακατασκευής εικόνων από το MNIST dataset, το οποίο περιλαμβάνει χειρόγραφους αριθμούς σε grayscale μορφή με διαστάσεις 28x28 pixels. Στη

συγκεκριμένη εφαρμογή, η πρόβλεψη του κάτω μισού αυτών των εικόνων βασίζεται στη λειτουργία ενός generator νευρωνικού δικτύου, το οποίο αποτελεί μέρος ενός GAN.

Η λειτουργία της εφαρμογής περιγράφεται ως εξής:

1. **Είσοδος δεδομένων:** Το σύστημα δέχεται ως είσοδο ένα υποσύνολο του MNIST dataset, όπου κάθε εικόνα έχει διαχωριστεί σε δύο μέρη. Ως είσοδος χρησιμοποιείται μόνο το πάνω μισό των εικόνων (14x28 pixels), το οποίο δίνεται στο νευρωνικό δίκτυο.

2. **Πρόβλεψη εξόδου:** Το generator του GAN επεξεργάζεται τα δεδομένα εισόδου και παράγει το κάτω μισό της εικόνας (14x28 pixels). Συνδυάζοντας τα δύο μέρη, προκύπτει η πλήρης ανακατασκευασμένη εικόνα.

3. **Αποθήκευση αποτελεσμάτων:** Τα αποτελέσματα της πρόβλεψης αποθηκεύονται σε ένα αρχείο εξόδου (output.txt). Το αρχείο περιλαμβάνει τις τιμές έντασης φωτεινότητας για κάθε pixel των ανακατασκευασμένων εικόνων σε μορφή τιμών από -1 έως 1.

4. **Αξιολόγηση απόδοσης:** Η εφαρμογή υλοποιείται τόσο σε λογισμικό (Software) όσο και σε υλικό (Hardware). Ο χρόνος εκτέλεσης μετράται σε κύκλους ρολογιού και συγκρίνεται για κάθε υλοποίηση. Το **speedup**, που υπολογίζεται ως ο λόγος των κύκλων του software προς τους κύκλους του hardware, χρησιμεύει ως δείκτης απόδοσης.

5. **Αξιολόγηση ποιότητας:** Ένα Jupyter Notebook παρέχεται για την οπτικοποίηση των αποτελεσμάτων. Συνδυάζοντας το πάνω μισό της εισόδου με την έξοδο του generator, το notebook εμφανίζει τις πλήρεις ανακατασκευασμένες εικόνες, επιτρέποντας τη σύγκριση μεταξύ των αποτελεσμάτων του software και του hardware.

Στόχος Βελτιστοποίησης

Η βασική πρόκληση της εφαρμογής εντοπίζεται στη συνάρτηση **forward_propagation**, η οποία αποτελεί το υπολογιστικά πιο απαιτητικό τμήμα του generator. Ο σκοπός είναι να μεταφερθεί η εκτέλεση αυτής της συνάρτησης στο FPGA του Zybo, αξιοποιώντας τις δυνατότητες του High-Level Synthesis (HLS). Οι τεχνικές HLS επιτρέπουν τη βελτιστοποίηση του κώδικα C, ώστε να μπορεί να υλοποιηθεί αποδοτικά σε υλικό (hardware).

Αναμενόμενα Αποτελέσματα

Η επιτυχής βελτιστοποίηση της συνάρτησης forward_propagation αναμένεται να μειώσει τον αριθμό των κύκλων ρολογιού που απαιτούνται για την εκτέλεση, επιτυγχάνοντας έτσι αύξηση του speedup. Επιπλέον, η ποιότητα της ανακατασκευής θα αξιολογηθεί για να διασφαλιστεί ότι η βελτιστοποίηση δεν επηρεάζει αρνητικά την ακρίβεια του μοντέλου.

Άσκηση 1. Performance and resources measurement

Εισαγωγή

Σκοπός της Άσκησης 1 είναι η βελτιστοποίηση του αλγορίθμου forward_propagation σε κώδικα C, ο οποίος αποτελεί τον generator του νευρωνικού δικτύου. Έτσι, κάνοντας χρήση του SDSoC και του High-Level Synthesis (HLS) επιδιώκουμε να αξιοποιήσουμε τα resources του Zybo για να πετύχουμε ταχύτερη εκτέλεση σε hardware υλοποίηση. Καλούμαστε, λοιπόν, να κάνουμε Design Space Exploration, δοκιμάζοντας διάφορα **HLS pragmas** στον κώδικα του αλγορίθμου και συγκρίνοντας το **performance estimation** με την αρχική (unoptimized) υλοποίηση. Έτσι, όταν είμαστε ικανοποιημένοι με το estimation, φτιάχνουμε bitstream και τρέχουμε την εφαρμογή στο board, καταγράφοντας τους **κύκλους** και το **speed-up** σε σχέση με την software εκτέλεση στον ARM. Τέλος, αναλύουμε το HLS report της τελικής (optimized) υλοποίησης.

A) Αρχικά, θέτουμε την συνάρτηση forward_propagation ως HW function στο SDSoC και κάνουμε estimate performance στην αρχική (**unoptimized**) υλοποίηση. Για να γίνει αυτό κάνουμε untick τις επιλογές Generate bitstream και Generate SD card image και κάνουμε tick την επιλογή Estimate performance. Το αποτέλεσμα που παίρνουμε φαίνεται παρακάτω:

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	683780

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	3	80	3,75
BRAM	16	60	26,67
LUT	1760	17600	10
FF	892	35200	2,53

Παρατηρούμε ότι η χρήση πόρων του FPGA είναι πολύ μικρή, πράγμα που σημαίνει ότι υπάρχει μεγάλο περιθώριο για να δοκιμάσουμε και βελτιώσουμε την υλοποίηση του HW function.

B) Έπειτα, θέλουμε να παράξουμε bitstream για να συγκρίνουμε τους HW κύκλους σε σχέση με το estimation που έγινε προηγουμένως και να δούμε το speed-up σε σχέση με την SW εκτέλεση στον ARM. Έτσι, κάνουμε untick την επιλογή Estimate performance και κάνουμε tick τις επιλογές Generate bitstream και Generate SD card image. Βάζουμε την SD card στο Zybo, κάνουμε επανεκκίνηση και τρέχουμε το αρχείο embedded.elf που βρίσκεται στο directory /mnt. Το αποτέλεσμα εκτέλεσης του κώδικα που παίρνουμε φαίνεται παρακάτω:

```
sh-4.3# cd mnt
sh-4.3# ls
BOOT.BIN                    _sds                          image.ub
README.txt                  data.txt
System Volume Information   embedded.elf
sh-4.3# ./embedded.elf
Starting dataset parsing...
Parsing finished...
Starting hardware calculations...
Hardware calculations finished.
Starting software calculations...
Software calculations finished.
Hardware cycles : 683015
Software cycles : 1474028
Speed-Up        : 2.15812
Saving results to output.txt...
sh-4.3#
```

Παρατηρούμε ότι ο αριθμός των HW cycles είναι πολύ κοντά σε αυτόν που προέκυψε από το estimation του SDSoC. Επίσης, βλέπουμε ότι το speed-up που πετυχαίνει η εφαρμογή χωρίς **καμία βελτιστοποίηση** είναι **2.15812.**

Γ) Σε αυτό το ερώτημα μας ζητείται να πραγματοποιήσουμε Design Space Exploration για να βρούμε τα optimizations που θα πετύχουν την μέγιστη επιτάχυνση του αλγορίθμου. Από εδώ και πέρα θα αναφέρουμε ποια HLS pragmas χρησιμοποιήσαμε και τι αποτέλεσμα επέφεραν στους εκτιμώμενους κύκλους της hardware υλοποίησης, κάνοντας κάθε φορά Estimate performance.

- pipeline II=1 στα απλά και στα εσωτερικά loops

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	221487

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	4	80	5
BRAM	16	60	26,67
LUT	1897	17600	10,78
FF	881	35200	2,5

Παρατηρούμε ότι ο αριθμός των κύκλων ελαττώθηκε σημαντικά (~67.61%) σε σχέση με την αρχική υλοποίηση ενώ ταυτόχρονα δεν υπάρχει σημαντική αύξηση στα resources.

- pipeline II=1 στα απλά, στα εσωτερικά και στα εξωτερικά loops

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	86183

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	41	60	68,33
LUT	6037	17600	34,3
FF	6754	35200	19,19

Παρατηρούμε ότι ο αριθμός των κύκλων ελαττώθηκε σημαντικά (~84.4%) σε σχέση με την αρχική υλοποίηση αλλά ταυτόχρονα σημειώνεται σημαντική αύξηση στα resources.

- unroll factor=2 στα απλά και στα εσωτερικά loops

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	427953

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	10	80	12,5
BRAM	17	60	28,33
LUT	2448	17600	13,91
FF	1087	35200	3,09

- unroll factor=4 στα απλά και στα εσωτερικά loops

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	405600

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	8	80	10
BRAM	17	60	28,33
LUT	4264	17600	24,23
FF	1665	35200	4,73

- unroll factor=8 στα απλά και στα εσωτερικά loops

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	282509

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	24	80	30
BRAM	32	60	53,33
LUT	7291	17600	41,43
FF	2804	35200	7,97

Παρατηρούμε ότι loop unroll στα απλά και στα εσωτερικά loops, ακόμα και με factor=2, αν και σημειώνει σημαντική μείωση στους κύκλους, είναι χειρότερο σε σχέση με άλλες υλοποιήσεις. Ακόμη, μεγαλύτερα unroll factor, αν και λίγο καλύτερα, κοστίζουν πολύ παραπάνω σε πόρους.

- unroll factor=2 στα εσωτερικά loops και pipeline II=1 στα εσωτερικά και απλά loops

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	181207

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	10	80	12,5
BRAM	17	60	28,33
LUT	1978	17600	11,24
FF	969	35200	2,75

Μεγαλύτερο unroll factor δεν είχε κάποια βελτίωση στον αλγόριθμο. Επιπλέον, unroll στα απλά loops δεν μειώνει περαιτέρω του κύκλους, αλλά καταναλώνει και περισσότερα resources.

- unroll factor=2 στα εσωτερικά και pipeline II=1 στα εσωτερικά, εξωτερικά και απλά loops

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	86183

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	41	60	68,33
LUT	6037	17600	34,3
FF	6754	35200	19,19

Παρατηρούμε ότι ο αριθμός των κύκλων ελαττώθηκε σημαντικά (~84.4%) σε σχέση με την αρχική υλοποίηση αλλά ταυτόχρονα σημειώνεται σημαντική αύξηση στα resources.

- unroll factor=2 στο εξωτερικό loop του layer 1 μαζί με τα προηγούμενα pragmas

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	47963

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	41	60	68,33
LUT	7735	17600	43,95
FF	7456	35200	21,18

- unroll factor=2 στο εξωτερικό loop του layer 2 αντί για το layer 1

Details

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	86021

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	72	60	120
LUT	7869	17600	44,71
FF	7742	35200	21,99

- unroll factor=2 στο εξωτερικό loop του layer 3 αντί για το layer 1

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	86183

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	266	60	443,33
LUT	10516	17600	59,75
FF	7269	35200	20,65

Παρατηρούμε ότι μόνο όταν κάνουμε unroll το εξωτερικό loop του layer 1 έχουμε έγκυρη υλοποίηση. Συγκεκριμένα έχουμε σημαντική ελάττωση των κύκλων (~92.99%) σε σχέση με την αρχική υλοποίηση και πολύ μεγαλύτερη αξιοποίηση των πόρων. Κάνοντας unroll οποιοδήποτε άλλο layer, καθώς και συνδυασμό αυτών, οι απαιτήσεις σε BRAM ξεπερνάνε σημαντικά τους διαθέσιμους πόρους του FPGA και οδηγούν σε σφάλμα.

- unroll factor=4 στο εξωτερικό loop του layer 1 μαζί με τα προηγούμενα pragmas

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	28853

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	41	60	68,33
LUT	8819	17600	50,11
FF	8515	35200	24,19

- unroll factor=8 στο εξωτερικό loop του layer 1 μαζί με τα προηγούμενα pragmas

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	24076

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	41	60	68,33
LUT	12870	17600	73,12
FF	10968	35200	31,16

- unroll factor=10 στο εξωτερικό loop του layer 1 μαζί με τα προηγούμενα pragmas

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	11115

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	41	60	68,33
LUT	13725	17600	77,98
FF	10321	35200	29,32

- unroll factor=12 στο εξωτερικό loop του layer 1 μαζί με τα προηγούμενα pragmas

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	11115

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	41	60	68,33
LUT	14730	17600	83,69
FF	10887	35200	30,93

Παρατηρούμε ότι η αύξηση του unroll factor στο εξωτερικό loop του layer 1 προκαλεί με την σειρά του περαιτέρω ελάττωση των κύκλων αλλά και μεγαλύτερη κατανάλωση πόρων, και πιο συγκεκριμένα των LUTs. Συνεπώς, το μεγαλύτερο unroll factor που έχει ως αποτέλεσμα έγκυρη υλοποίηση και δεν ξεπερνάει το όριο των διαθέσιμων LUTs είναι το 12. Αυτή η υλοποίηση έχει σημαντική ελάττωση κύκλων (~98.37%) και αξιοποιεί πολύ ικανοποιητικά τα resources.

Ανοίγοντας, όμως το HLS Report της υλοποίησής μας βλέπουμε τα παρακάτω:

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	8.70	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
1153	1153	1154	1154	none

Detail

Instance

Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- read_input	395	395	5	1	1	392	yes
- layer_1	212	212	9	7	1	30	yes
- layer_1_act	30	30	2	1	1	30	yes
- layer_2	52	52	4	1	1	50	yes
- layer_3	400	400	10	1	1	392	yes

Παρατηρούμε ότι το design μας δεν είναι pipelined (με Initiation Interval=1), καθώς, με την παρούσα υλοποίηση που έχουμε κάνει, το layer 1 έχει initiation interval=7 που είναι σχεδόν όσο και ο χρόνος ενός iteration (Iteration Latency =9) του loop. Επομένως, πρέπει να βρούμε μια διαφορετική προσέγγιση ώστε το πρόγραμμά μας να είναι πλήρως pipelined, χωρίς όμως να έχουμε μεγάλη αύξηση των HW cycles.

Ανοίγουμε το αρχείο `network.h` και βλέπουμε ότι τα όρια των εσωτερικών for loop σε κάθε layer είναι σχετικά μικροί αριθμοί. Έτσι, αποφασίζουμε να κάνουμε loop unroll με μέγιστο factor σε όλα τα εσωτερικά for loops κάθε layer και ταυτόχρονα να κάνουμε pipeline σε όλα τα εξωτερικά for loops. Επιπλέον, με δοκιμές διαπιστώσαμε ότι μπορούμε να εφαρμόσουμε loop unroll με factor =2 και στο εξωτερικό for loop του layer 1. Η υλοποίηση που προκύπτει είναι η παρακάτω:

- max unroll factor σε εσωτερικά loops, pipeline II=1 σε εξωτερικά loops και unroll factor=2 στο εξωτερικό loop του layer 1

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	10718

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	41	60	68,33
LUT	7587	17600	43,11
FF	5996	35200	17,03

- array partition στους πίνακες layer 1 out και layer 2 out μαζί με τα προηγούμενα pragmas

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	10556

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	41	60	68,33
LUT	7446	17600	42,31
FF	6008	35200	17,07

Παρατηρούμε ότι έχουμε ακόμη μεγαλύτερη μείωση των κύκλων σε σχέση με τις προηγούμενες υλοποιήσεις και πολύ καλή αξιοποίηση των διαθέσιμων resources. Η συνολική ελάττωση σε κύκλους ανέρχεται σε ~98.46%. Καθώς είμαστε ικανοποιημένοι με το παραπάνω design, αυτή είναι η τελική (**optimized**) υλοποίηση, την οποία θα δοκιμάσουμε να τρέξουμε στο Zybo. Έτσι, κάνουμε build project, έχοντας κάνει tick τις επιλογές Generate bitstream και Generate SD card image, προκειμένου να βγάλουμε το απαραίτητο bitstream. Έπειτα, βάζουμε την SD card στο Zybo, κάνουμε επανεκκίνηση και τρέχουμε το αρχείο embedded.elf που βρίσκεται στο directory /mnt. Το αποτέλεσμα εκτέλεσης του κώδικα που παίρνουμε φαίνεται παρακάτω:

```
sh-4.3# cd mnt/
sh-4.3# ./embedded.elf
Starting dataset parsing...
Parsing finished...
Starting hardware calculations...
Hardware calculations finished.
Starting software calculations...
Software calculations finished.
Hardware cycles : 10780
Software cycles : 1473904
Speed-Up        : 136.726
Saving results to output.txt...
sh-4.3#
```

Παρατηρούμε ότι ο αριθμός των HW cycles είναι πολύ κοντά σε αυτόν που προέκυψε από το estimation του SDSoC. Επίσης, βλέπουμε ότι το speed-up που πετυχαίνει η εφαρμογή αυτή την φορά με **όλες τις βελτιστοποιήσεις** είναι **136.726**, ένας αριθμός που είναι πάνω από **63 φορές** μεγαλύτερος από το speed-up που προέκυψε στην αρχική υλοποίηση του αλγορίθμου.

Δ) Σε αυτό το ερώτημα μας ζητείται να εξετάσουμε το HLS Report για την βέλτιστη υλοποίηση που πραγματοποιήσαμε στο προηγούμενο ερώτημα. Έτσι, έχουμε το παρακάτω report:

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	8.70	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
1052	1052	1053	1053	none

Detail

Instance

Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- read_input	395	395	5	1	1	392	yes
- layer_1	197	197	3	1	1	196	yes
- layer_2	52	52	4	1	1	50	yes
- layer_3	400	400	10	1	1	392	yes

Παρατηρούμε από το report ότι το design μας είναι πλήρως pipelined, καθώς σε κάθε επίπεδο έχουμε Initiation Interval = 1. Ακόμη, μεγαλύτερο latency στην υλοποίησή μας παρουσιάζει το layer 3 με Iteration Latency = 10.

Έπειτα μας ζητείται να εξετάσουμε και την καρτέλα Resource Profile του HLS Report. Καθώς εμάς μας ενδιαφέρει η καταγραφή των μαθηματικών εκφράσεων, πατάμε πάνω στο Expressions και παίρνουμε τον παρακάτω πίνακα:

	BRAM	DSP	FF	LUT	Bits P0	Bits P1	Bits P2	Banks/Depth	Words	W*Bits*Banks
forward_propagation	82	80	6008	7363						
I/O Ports(2)					64					
Instances(2)	0	0	268	677						
Memories(112)	82		273	232	1016			112	34276	310594
Expressions(504)	0	80	0	4669	5063	4988	1156			
-	0	0	0	78	16	78	0			
*	0	80	0	0	1114	1005	0			
+	0	0	0	3054	3541	3530	0			
and	0	0	0	5	5	5	0			
ashr	0	0	0	161	54	54	0			
icmp	0	0	0	75	200	68	0			
or	0	0	0	20	16	8	0			
select	0	0	0	1167	72	191	1156			
shl	0	0	0	88	32	32	0			
xor	0	0	0	21	13	17	0			
Registers(524)			5467		6310					
Channels(0)	0		0	0	0			0	0	0
Multiplexers(134)	0		0	1785	1782			0		

Παρατηρούμε ότι και οι 80 μονάδες DSP χρησιμοποιούνται για πράξεις πολλαπλασιασμού. Αυτό είναι λογικό καθώς με τις αλλαγές που κάναμε στον δοσμένο αλγόριθμο, εκτελούνται πολύ περισσότερες πράξεις, επομένως υπάρχει ανάγκη να διατεθούν περισσότερες μονάδες εκτέλεσης υπολογισμών. Ακόμη, αξίζει να σημειωθεί ότι στην τελική (optimized) υλοποίηση όσον αφορά τα resources έχουμε **96.25% αύξηση σε DSPs, 60.98% αύξηση σε BRAMs, 76.24% αύξηση σε LUTs και 85.15% αύξηση σε FFs** σε σχέση με την αρχική (unoptimized) υλοποίηση.

Ο κώδικας την αρχικής υλοποίησης βρίσκεται στο αρχείο `unoptimized.c`.

Ο κώδικας την βέλτιστης υλοποίησης βρίσκεται στο αρχείο `optimized.c`.

Άσκηση 2. Quality measurement

Εισαγωγή

Σκοπός της Άσκησης 2 είναι η μέτρηση της ποιότητας της ανακατασκευής των εικόνων που παράγονται από το Generative Adversarial Network (GAN) της εφαρμογής, χρησιμοποιώντας τόσο τη λογισμική (SW) όσο και την υλική (HW) υλοποίηση. Η ανάλυση βασίζεται στον συνδυασμό της εισόδου (πάνω μισό της εικόνας) και της εξόδου (κάτω μισό της εικόνας) για διάφορα σενάρια ακρίβειας δεδομένων (bit-width). Οι παρατηρήσεις βασίζονται στις εικόνες που παράγονται για διαφορετικούς δείκτες (idx: 10, 11, 12), καθώς και στις τιμές **max pixel error** και **Peak Signal-to-Noise Ratio (PSNR)** που προκύπτουν.

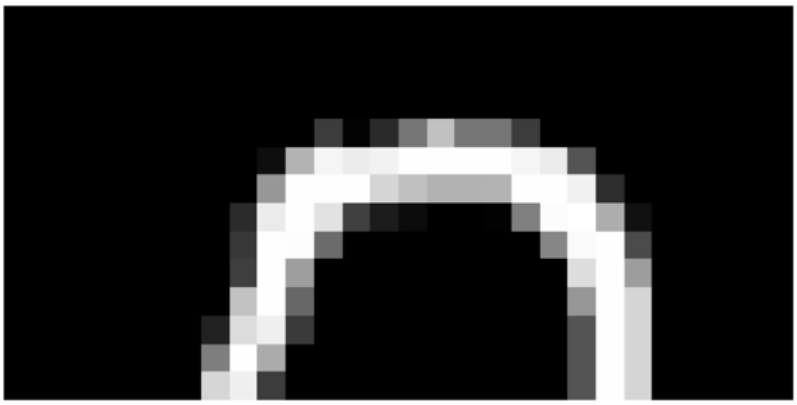
A) Χρησιμοποιώντας το αρχείο `plot_output.ipynb`, εμφανίστηκαν οι combined εικόνες για τους δείκτες idx: 10, 11, 12, τόσο για την έξοδο της λογισμικής υλοποίησης (SW) όσο και για την υλική υλοποίηση (HW). Μέχρι στιγμής η ακρίβεια των δομών δεδομένων ήταν 8 bits. Στην συνέχεια θα δοκιμάσουμε datatypes και με διαφορετικό αριθμό bit.

Για idx = 10, εμφανίζεται το πάνω μισό της εικόνας του αριθμού "0" από το MNIST dataset, το οποίο χρησιμοποιείται ως είσοδος στο σύστημα. Η εικόνα περιέχει μόνο τα πάνω 14x28 pixels του αριθμού, όπως καθορίζεται από το dataset.

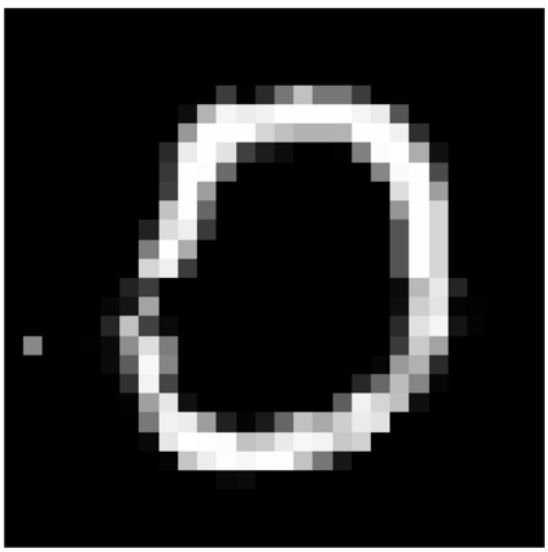
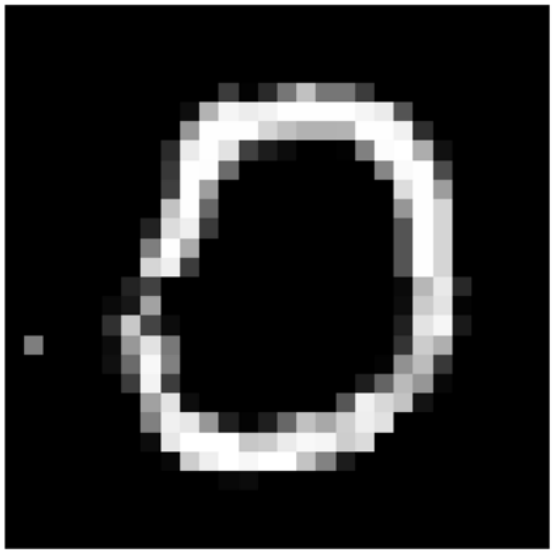
[7]:
```
#Set Image to evaluate
idx = 10

plt.figure()
plt.imshow(data[idx,:].reshape(14,28), cmap = 'gray')
plt.axis('off')
```

[7]:
```
(-0.5, 27.5, 13.5, -0.5)
```



Παρακάτω φαίνεται αριστερά η SW υλοποίηση και δεξιά την HW υλοποίηση:

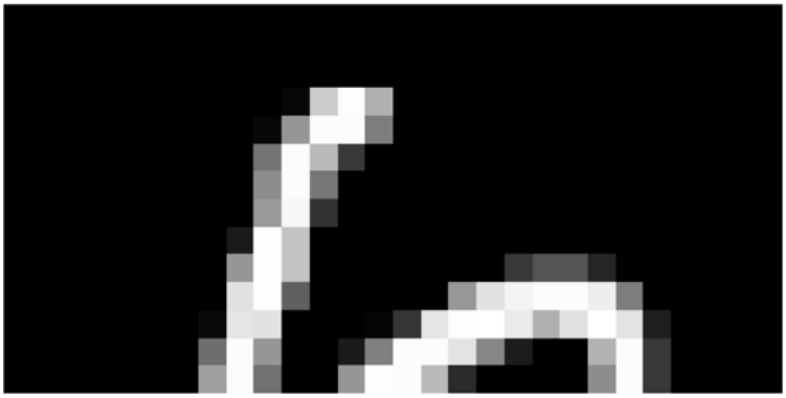


ε

Για idx = 11, εμφανίζεται το πάνω μισό της εικόνας του αριθμού "6" από το MNIST dataset, το οποίο χρησιμοποιείται ως είσοδος στο σύστημα.

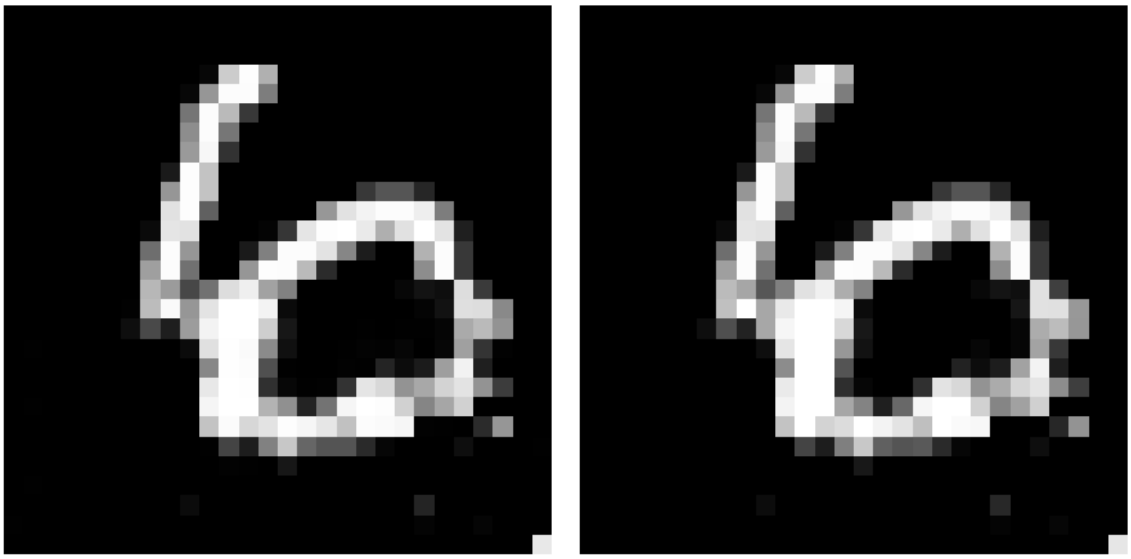
[29]:
```
#Set Image to evaluate
idx = 11

plt.figure()
plt.imshow(data[idx,:].reshape(14,28), cmap = 'gray')
plt.axis('off')
```

[29]:
```
(-0.5, 27.5, 13.5, -0.5)
```



Παρακάτω φαίνεται αριστερά η SW υλοποίηση και δεξιά την HW υλοποίηση:



Για idx = 12, εμφανίζεται το πάνω μισό της εικόνας του αριθμού "6" από το MNIST dataset, το οποίο χρησιμοποιείται ως είσοδος στο σύστημα.

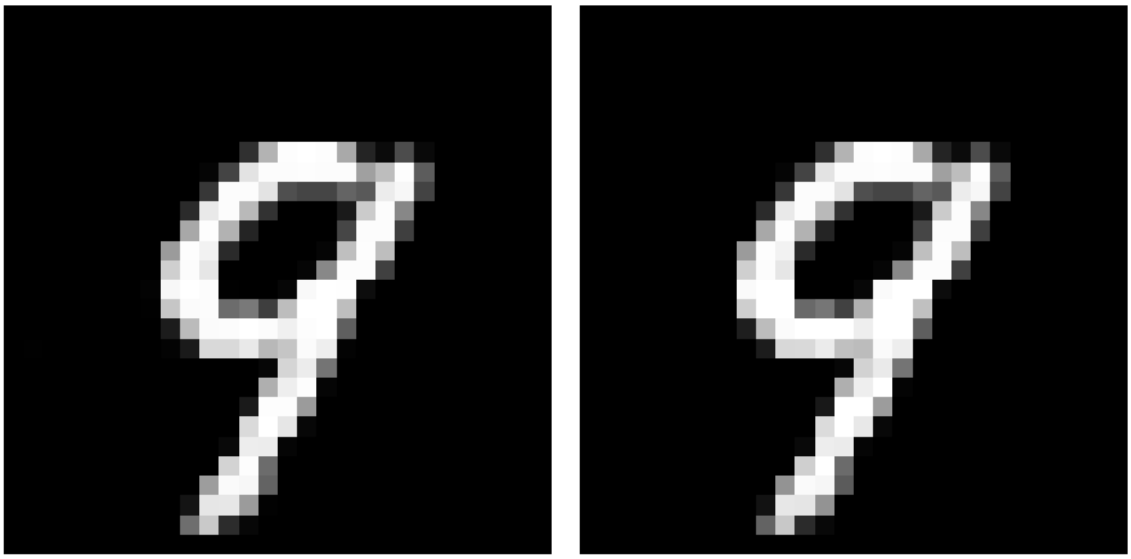
[37]:
```
#Set Image to evaluate
idx = 12

plt.figure()
plt.imshow(data[idx,:].reshape(14,28), cmap = 'gray')
plt.axis('off')
```

[37]:
```
(-0.5, 27.5, 13.5, -0.5)
```



Παρακάτω φαίνεται αριστερά η SW υλοποίηση και δεξιά την HW υλοποίηση:



Οι εικόνες που παράγονται με 8-bit ακρίβεια τόσο από τη λογισμική (SW) όσο και από την υλική (HW) υλοποίηση παρουσιάζουν πολύ μικρές διαφορές. Οπτικά, οι διαφορές είναι σχεδόν ανεπαίσθητες. Ο συνδυασμός του πάνω μισού της εικόνας (εισόδου) με το παραγόμενο κάτω μισό αποδεικνύει ότι το hardware μπορεί να προσεγγίσει το αποτέλεσμα του λογισμικού χωρίς εμφανείς απώλειες ποιότητας. Τα αποτελέσματα των μετρικών που χρησιμοποιούμε για την αξιολόγηση της ποιότητας των αναπαραστάσεων φαίνονται παρακάτω:

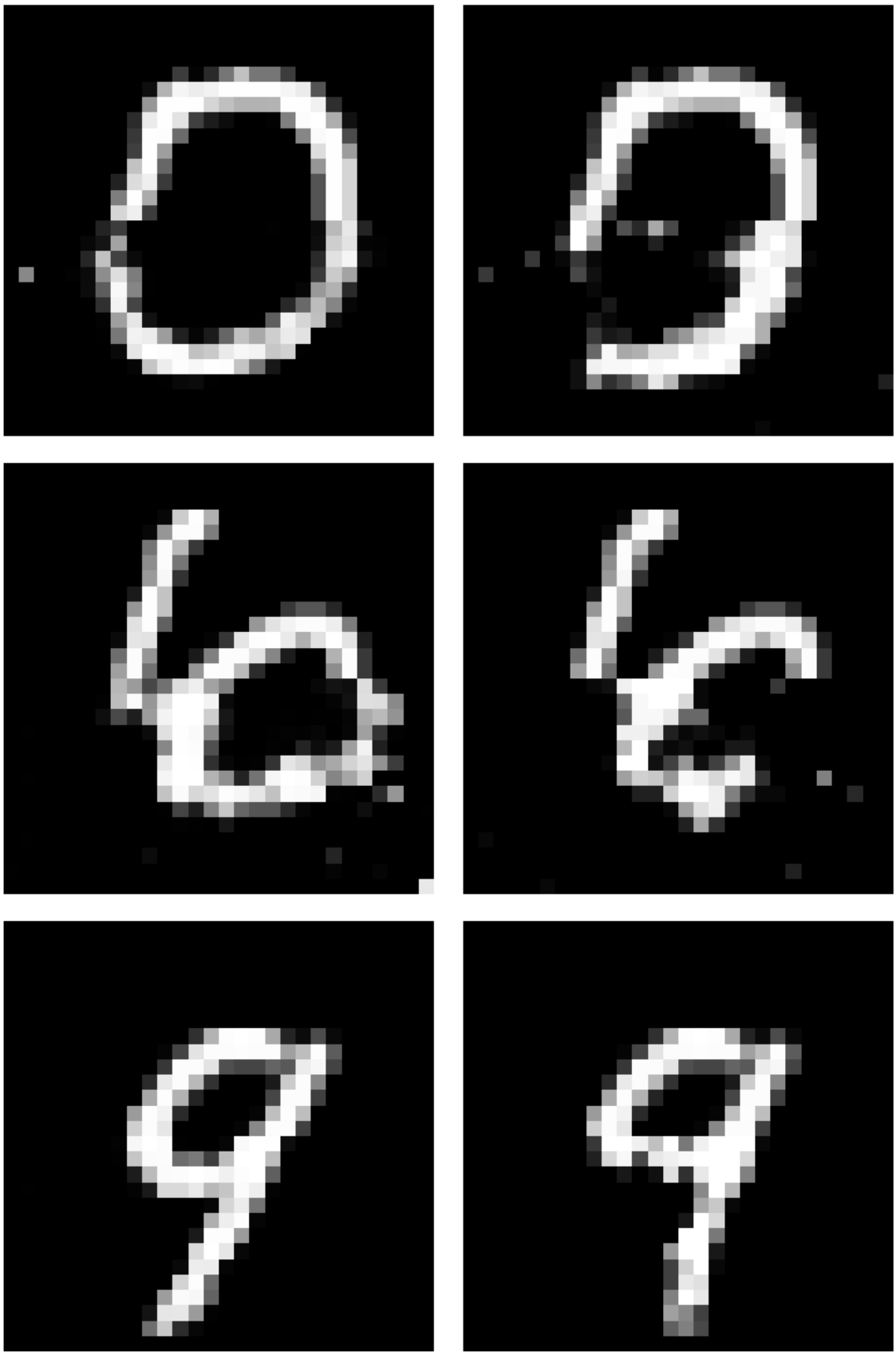
```
Max pixel error:  16
Peak Signal-to-Noise Ratio:  42.6822168370888

 Max pixel error:  17
 Peak Signal-to-Noise Ratio:  42.56993337396983

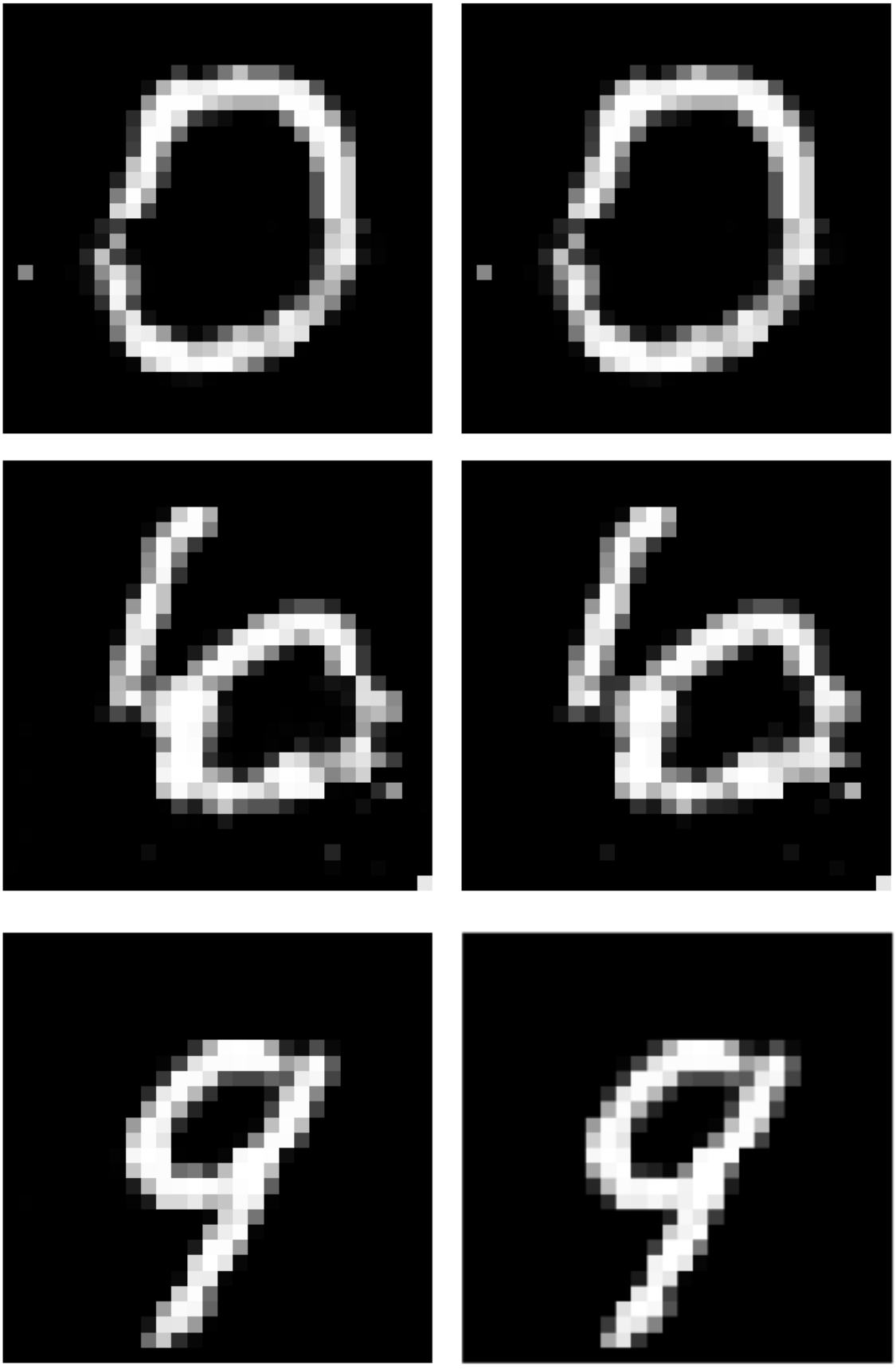
Max pixel error:  13
Peak Signal-to-Noise Ratio:  47.065287020211215
```

B) Με αλλαγή των παραμέτρων BITS και BITS_EXP στο αρχείο `network.h` και αντικατάσταση των pre-computed τιμών του tanh.h, δημιουργήθηκαν νέα σχέδια (designs) με ακρίβεια 4, 6 και 10 bits. Οι εικόνες που προέκυψαν για idx: 10, 11, 12 ήταν οι εξής:

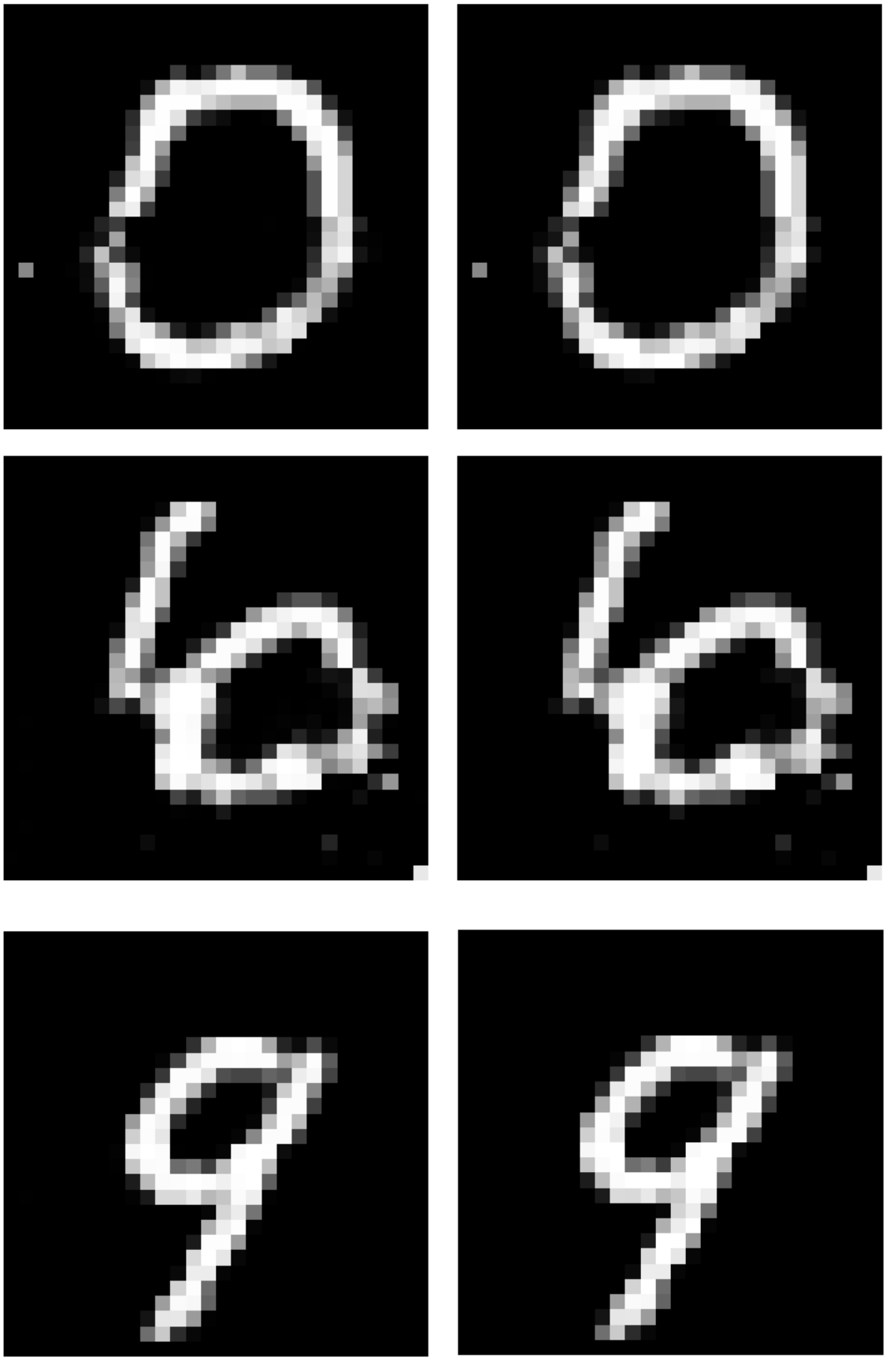
- **4 bits:** Οι εικόνες παρουσιάζουν εμφανή απώλεια λεπτομέρειας, με θόλωση και απόκλιση στη δομή των αριθμών. Οι γραμμές των ψηφίων είναι λιγότερο ευδιάκριτες, και η ποιότητα είναι σημαντικά υποδεέστερη.



- **6 bits:** Οι εικόνες πλησιάζουν πολύ περισσότερο αυτές της υλοποίησης με 8 bits, παρουσιάζοντας καλύτερη ευκρίνεια και λεπτομέρεια. Οι διαφορές με το SW είναι πολύ μικρότερες σε σχέση με την ακρίβεια 4 bits, αλλά ακόμα διακρίνουμε ατέλειες.



- **10 bits:** Οι εικόνες πλέον δεν έχουν ευδιάκριτες διαφορές και η ανάλυση και η ακρίβεια των εικόνων πλησιάζει και ξεπερνάει την υλοποίηση των 8-bit.



Παρατηρούμε ότι η ποιότητα της ανακατασκευής εξαρτάται έντονα από την ακρίβεια των bits που χρησιμοποιούνται. Συγκεκριμένα:

- **Για 4-bit ακρίβεια:** Οι εικόνες εμφανίζουν αισθητή υποβάθμιση, με πολλές ασυνέχειες και θολά σημεία. Αυτό συμβαίνει επειδή η χαμηλή ακρίβεια δεν μπορεί να αποδώσει σωστά τις ενδιάμεσες τιμές των pixels, με αποτέλεσμα να "τραβιούνται" σε ακραίες τιμές (π.χ., κοντά στο 0 ή στο 1). Η εικόνα γίνεται λιγότερο ρεαλιστική, ειδικά στις περιοχές με ενδιάμεσες φωτεινότητες.

- **Για 6-bit ακρίβεια:** Παρατηρούμε σαφή βελτίωση σε σχέση με τα 4 bits. Οι λεπτομέρειες είναι πιο ευδιάκριτες και οι αριθμοί φαίνονται πιο σωστά ανακατασκευασμένοι. Παρ' όλα αυτά, στις περιοχές όπου τα pixels έχουν ενδιάμεσες τιμές φωτεινότητας, η διαχείριση τους παραμένει προβληματική. Τα pixels εξακολουθούν να τείνουν σε ακραίες τιμές, γεγονός που δημιουργεί ασυνέχειες, αν και αυτές είναι λιγότερο εμφανείς σε σχέση με την περίπτωση των 4 bits.

- **Για 8-bit ακρίβεια:** Οι εικόνες παρουσιάζουν πολύ υψηλή ποιότητα, χωρίς εμφανείς ασυνέχειες ή απώλεια λεπτομερειών. Τα pixels με ενδιάμεσες τιμές φωτεινότητας αποδίδονται με μεγαλύτερη ακρίβεια, προσφέροντας ένα αποτέλεσμα που είναι σχεδόν πανομοιότυπο με το λογισμικό.

- **Για 10-bit ακρίβεια:** Οι εικόνες πλησιάζουν ακόμα περισσότερο την ακρίβεια του λογισμικού, με ελάχιστες έως καθόλου διαφορές, ακόμα και σε περιοχές με ενδιάμεσες τιμές φωτεινότητας. Η ανακατασκευή είναι εξαιρετικά ομαλή και λεπτομερής.

Η εικόνα που παράγεται από τη λογισμική υλοποίηση (SW) αποτελείται κυρίως από pixels με τιμές πολύ κοντά στο 0 ή στο 1. Στην υλοποίηση σε hardware (HW), όταν η ακρίβεια των bits είναι χαμηλή, τα pixels με ενδιάμεσες τιμές φωτεινότητας μετατρέπονται σε ακραίες τιμές (0 ή 1). Αυτό δημιουργεί ομοιότητες στις περιοχές όπου τα pixels είναι οριακά, αλλά προκαλεί σημαντικές ασυνέχειες στις περιοχές με πολλές ενδιάμεσες τιμές. Αυτό είναι ιδιαίτερα εμφανές σε περιπτώσεις με περισώτερες λεπτομέρειες όπως οι αριθμοί "0" και "6", όπου οι περιοχές με ενδιάμεσες τιμές φωτεινότητας είναι πιο διαδεδομένες.

Αυτές οι ασυνέχειες μειώνονται καθώς αυξάνεται η ακρίβεια των bits, επιτρέποντας καλύτερη απόδοση των ενδιάμεσων τιμών και συνεπώς πιο ομαλές και ρεαλιστικές εικόνες.

Γ) Οι μετρήσεις ποιότητας ανακατασκευής που υπολογίστηκαν στο `plot_output.ipynb` για διαφορετικά bit-widths είναι οι εξής:

4-Bits			
IDX	10	11	12
Max Pixel Error	255	249	255
Peak Signal-to-Noise Ratio	14.05166	14.63499	13.52583
6-Bits			
IDX	10	11	12
Max Pixel Error	91	98	93
Peak Signal-to-Noise Ratio	28.15987	27.496646	19.2355714
8-Bits			
IDX	10	11	12
Max Pixel Error	16	17	13
Peak Signal-to-Noise Ratio	42.68222	42.569933	27.06529
10-Bits			
IDX	10	11	12
Max Pixel Error	5	5	4
Peak Signal-to-Noise Ratio	54.085433	52.5561	53.76983

- **Max Pixel Error:** Το **Max Pixel Error** είναι μια μετρική που εκφράζει τη μέγιστη απόκλιση ανά pixel μεταξύ δύο εικόνων, συνήθως της ανακατασκευασμένης εικόνας (HW) και της αναφοράς (SW).
    - Είναι μια πολύ άμεση μετρική που υποδεικνύει τη μεγαλύτερη απόκλιση σε οποιοδήποτε pixel. Ωστόσο, επηρεάζεται από μεμονωμένα pixels με ακραίες διαφορές, ακόμα κι αν το υπόλοιπο της εικόνας είναι αρκετά κοντά στο ιδανικό.

    - Όσο αυξάνονται τα bits ακρίβειας, η απόκλιση αυτή μειώνεται, καθώς οι υπολογισμοί γίνονται πιο ακριβείς και αποφεύγεται η στρογγυλοποίηση τιμών (quantization error)
- **PSNR:** Το **PSNR (Peak Signal-to-Noise Ratio)** είναι ένας πιο ολοκληρωμένος δείκτης που μετρά την ποιότητα της ανακατασκευασμένης εικόνας σε σχέση με την εικόνα αναφοράς.
    - Το PSNR λαμβάνει υπόψη όλες τις αποκλίσεις στην εικόνα, δίνοντας έμφαση στο μέσο όρο των σφαλμάτων, αντί για το μέγιστο όπως στο Max Pixel Error.
    - **Υψηλές τιμές = Καλύτερη ποιότητα:** Υψηλότερο PSNR υποδεικνύει ότι η ανακατασκευασμένη εικόνα είναι πιο κοντά στην εικόνα αναφοράς. Συνήθως, PSNR > 30 dB θεωρείται καλή ποιότητα για εικόνες.
    - Είναι πιο ευαίσθητο στις συνολικές αλλαγές της εικόνας, παρέχοντας μια πιο αντικειμενική μέτρηση της ποιότητας.

Προτίμηση Τεχνικής: Το PSNR είναι πιο κατάλληλο ως μέτρο ποιότητας, καθώς παρέχει μια γενική εκτίμηση της σχέσης μεταξύ του σήματος και του θορύβου, λαμβάνοντας υπόψη το συνολικό αποτέλεσμα της εικόνας και δεν επηρεάζεται από μεμονωμένες μεγάλες αποκλίσεις.

Κοιτάζοντας τα παραπάνω αποτελέσματα, επιβεβαιώνουμε “επιστημονικά” τα εμπειρικά συμπεράσματα που προέκυψαν από την παρατήρηση των εικόνων στις διάφορες ακρίβειες. Παρατηρούμε πως η αύξηση της ακρίβειας των bits οδηγεί σε σημαντική βελτίωση της ποιότητας της ανακατασκευασμένης εικόνας, γεγονός που αντικατοπτρίζεται και στις μετρικές Max Pixel Error και PSNR.

Ακρίβεια 4-bits

- Max Pixel Error: Το σφάλμα φτάνει τη μέγιστη δυνατή τιμή (255) σε ορισμένα pixels, κάτι που δείχνει ότι η χαμηλή ακρίβεια προκαλεί μεγάλες αποκλίσεις. Τα pixels με ενδιάμεσες τιμές φωτεινότητας δεν αποδίδονται σωστά, δημιουργώντας έντονες ασυνέχειες.
- PSNR: Οι τιμές PSNR (13.5-14.6 dB) είναι πολύ χαμηλές, υποδεικνύοντας υποβαθμισμένη συνολική ποιότητα εικόνας. Η χαμηλή ακρίβεια δεν είναι κατάλληλη για εφαρμογές που απαιτούν υψηλή πιστότητα εικόνας.

Ακρίβεια 6-bits

- Max Pixel Error: Το σφάλμα μειώνεται σημαντικά (91-98), αλλά εξακολουθεί να είναι υψηλό, ιδιαίτερα στις περιοχές με ενδιάμεσες φωτεινότητες. Παρά τη βελτίωση, οι ασυνέχειες παραμένουν εμφανείς σε αυτές τις περιοχές.
- PSNR: Οι τιμές PSNR (19.2-28.1 dB) δείχνουν αισθητή βελτίωση σε σχέση με τα 4 bits, ωστόσο η ποιότητα παραμένει μέτρια. Η ακρίβεια αυτή προσφέρει καλύτερη ισορροπία

μεταξύ απόδοσης και απαιτήσεων, αλλά εξακολουθεί να μην είναι κατάλληλη για εφαρμογές υψηλής ποιότητας.

Ακρίβεια 8-bits

- Max Pixel Error: Το σφάλμα μειώνεται περαιτέρω (13-17), κάτι που υποδηλώνει ότι τα περισσότερα pixels είναι αρκετά κοντά στις σωστές τιμές. Οι ενδιάμεσες φωτεινότητες αποδίδονται πλέον με μεγάλη ακρίβεια.
- PSNR: Οι τιμές PSNR (27.0-42.6 dB) δείχνουν ότι η συνολική ποιότητα εικόνας είναι πολύ καλή. Η ακρίβεια 8 bits προσφέρει ένα εξαιρετικό επίπεδο ποιότητας, ικανοποιώντας τις περισσότερες απαιτήσεις σε πιστότητα εικόνας, με παράλληλα λογική κατανάλωση πόρων.

Ακρίβεια 10-bits

- Max Pixel Error: Το σφάλμα μειώνεται περαιτέρω (4-5), επιτυγχάνοντας σχεδόν ιδανική ακρίβεια. Τα pixels αποδίδονται σχεδόν τέλεια, με μικρές μόνο αποκλίσεις.
- PSNR: Οι τιμές PSNR (52.5-54.0 dB) δείχνουν εξαιρετική ποιότητα εικόνας, που είναι σχεδόν πανομοιότυπη με την ανακατασκευή του λογισμικού. Παρόλο που η ποιότητα είναι ελαφρώς βελτιωμένη σε σχέση με τα 8 bits, η διαφορά αυτή είναι μικρή και πιθανόν δυσδιάκριτη στο ανθρώπινο μάτι.

Η αύξηση της ακρίβειας των bits έχει σαφώς θετική επίδραση στην ποιότητα της ανακατασκευής, με σχεδόν υπερ-γραμμική βελτίωση στις μετρικές. Ωστόσο, η αύξηση των bits οδηγεί και σε υψηλότερη κατανάλωση πόρων, δημιουργώντας ένα trade-off μεταξύ quality of service (QoS) και πολυπλοκότητας.

- Η ακρίβεια 4-bits είναι η πιο “ελαφριά” λύση, αλλά δίνει χαμηλή ποιότητα και δεν είναι κατάλληλη για εφαρμογές που απαιτούν υψηλή πιστότητα.
- Η ακρίβεια 6-bits προσφέρει βελτίωση, αλλά εξακολουθεί να είναι αισθητά κατώτερη των 8 bits.
- Η ακρίβεια 8-bits είναι η καλύτερη ισορροπία μεταξύ ποιότητας και πόρων, προσφέροντας πολύ καλή απόδοση χωρίς υπερβολικές απαιτήσεις.
- Η ακρίβεια 10-bits δίνει την καλύτερη δυνατή ποιότητα, αλλά το κόστος σε πόρους και πολυπλοκότητα καθιστά τη χρήση της λιγότερο αποδοτική για τις περισσότερες πρακτικές εφαρμογές.

Η ακρίβεια 8 bits είναι η πιο ισορροπημένη επιλογή, καθώς προσφέρει ικανοποιητική ποιότητα (υψηλό PSNR) με αποδεκτό κόστος σε υπολογιστικούς πόρους. Αντίθετα, η ακρίβεια 10 bits, αν και προσφέρει ελαφρώς καλύτερη ποιότητα, αυξάνει το κόστος σε hardware resources, ενώ η ακρίβεια 4 ή 6 bits υποβαθμίζει σημαντικά την ποιότητα.